The image shows a dropdown list of length units with “Meter” highlighted: Kilometer, Meter, Centimeter, Millimeter, Micrometer, Nanometer, Mile, Yard, Foot, and Inch.

There’s no question or message with it, so I’m not sure what you’d like to do. A few possibilities:

- **Unit conversion**: if you’re converting between these units, tell me the value and the units, and I can work it out.
- **UI feedback**: if this is a dropdown you’re designing, I can suggest improvements (for example, grouping metric and imperial units, or ordering by size).
- **Code help**: if you’re building a converter or this dropdown itself, share the code or language and I can help.

What are you trying to do?

meter_dict = { "kilometer": 0.001,
 "": 0.0}
length_dictionary = {}

Completing the meter factors in a unit converter.

It looks like you’re building a unit converter. Your `"kilometer": 0.001` entry means 1 meter = 0.001 km, so each value is “how many of this unit equal one meter.” Here’s the dictionary completed for every unit in your dropdown, with a conversion function:

python

```python
# How many of each unit are in 1 meter
meter_dict = {
    "kilometer":  0.001,
    "meter":      1.0,
    "centimeter": 100.0,
    "millimeter": 1_000.0,
    "micrometer": 1_000_000.0,
    "nanometer":  1_000_000_000.0,
    "mile":       1 / 1609.344,
    "yard":       1 / 0.9144,
    "foot":       1 / 0.3048,
    "inch":       1 / 0.0254,
}

def convert(value, from_unit, to_unit):
```

```
    from_unit, to_unit = from_unit.lower(), to_unit.lower()
    in_meters = value / meter_dict[from_unit]   # convert to meters
first
    return in_meters * meter_dict[to_unit]      # then to the
target unit

print(convert(1, "mile", "kilometer"))   # ~1.609344
print(convert(12, "inch", "foot"))       # 1.0
```

A few notes:

- **Why go through meters:** with one base unit, you only need N entries instead of an N×N table of conversions.
- **Your empty `""` key:** I dropped it, since an empty-string unit name would never match a dropdown selection.
- **`length_dictionary`:** if you meant this to be a second dictionary, tell me what it was for. If it was meant to hold the same data, you can just use `meter_dict`.
- **Dropdown matching:** the dropdown shows capitalized names like “Meter”, so the `.lower()` calls make lookups match your lowercase keys.

If you’d like, I can wire this to a GUI (Tkinter, for example) so the dropdown drives the conversion.

Can you make a dictionary for every possible conversion between each?

Writing out all 100 pairs by hand would be error-prone, so it’s better to generate the dictionary from one list of base values. That gives you a nested dictionary where `conversions[from_unit][to_unit]` is the number to multiply by:

python

Python
```
# How many meters are in 1 of each unit (exact definitions)
meters_per_unit = {
    "kilometer":  1000.0,
    "meter":      1.0,
    "centimeter": 0.01,
    "millimeter": 0.001,
    "micrometer": 1e-6,
```

```
    "nanometer":  1e-9,
    "mile":       1609.344,
    "yard":       0.9144,
    "foot":       0.3048,
    "inch":       0.0254,
}

# Every possible conversion: conversions[from][to] = multiplier
conversions = {
    from_unit: {
        to_unit: from_m / to_m
        for to_unit, to_m in meters_per_unit.items()
    }
    for from_unit, from_m in meters_per_unit.items()
}

def convert(value, from_unit, to_unit):
    return value * conversions[from_unit.lower()][to_unit.lower()]

print(conversions["mile"]["kilometer"])   # 1.609344
print(conversions["foot"]["inch"])        # 12.0 (may show tiny
float error)
print(convert(5, "Yard", "Meter"))        # 4.572
```

This builds all 100 entries (including each unit to itself, which is 1.0), and it's correct by construction, since every value is derived from the same base table. If you add a new unit later, you only add one line to `meters_per_unit` and the whole table updates.

Float math can leave tiny errors like `12.000000000000002`. If that matters for display, wrap results in `round(x, 10)`, or use Python's `decimal` or `fractions` module for exact results.

If you'd rather have the full 10×10 dictionary written out literally as hardcoded numbers, I can generate that for you to paste in.

Including between each other

The nested dictionary I gave already covers every pair, including metric to imperial and back (for example `conversions["mile"]["inch"]` and `conversions["nanometer"]["yard"]`). Since you may want it written out explicitly, I'll generate the full literal version.

I've written `conversions.py`, which has the full 10×10 dictionary with all 100 conversions hardcoded, including every unit to every other unit. It includes a `convert()` function, and a test run gave the expected result (1 mile = 63,360 inches).

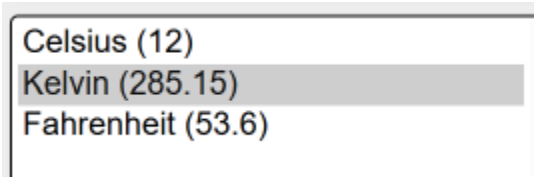
The values were computed from exact definitions (for example, 1 inch = 0.0254 m exactly) and then written out as plain numbers. You can paste the dictionary straight into your project without the generator code.

Using it looks like this:

python

```python
convert(5, "Yard", "Meter")      # 4.572
convert(1, "Mile", "Kilometer")  # 1.609344
```

Conversions
PY



Do the same with this:

Listing temperature conversions and planning a dictionary of them.

Temperature differs from length because conversions need an offset as well as a multiplier, so each entry is a function instead of a single number. I'll write all 9 pairs explicitly and test them against the values in your screenshot (12 °C, 285.15 K, 53.6 °F).

I've written `temperature_conversions.py`, which has all 9 temperature conversions (including each unit to itself) as a nested dictionary of functions. All six cross-conversions reproduce the values in your screenshot: 12 °C = 285.15 K = 53.6 °F.

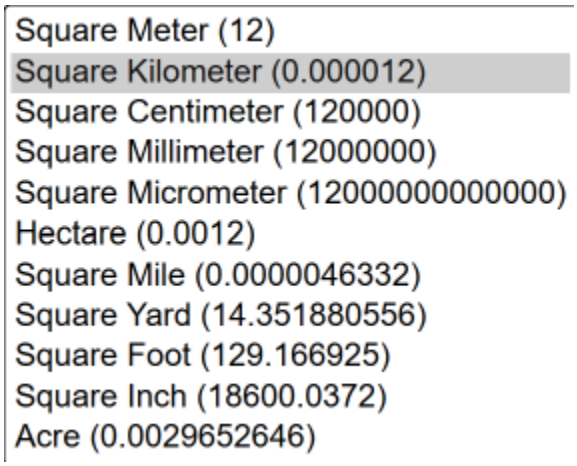
You call it the same way as the length converter:

python

```python
convert(12, "Celsius", "Kelvin")      # 285.15
convert(53.6, "Fahrenheit", "Celsius")  # 12.0
```

If your app has both dropdowns, you can merge the two files into one structure, such as `{"length": ..., "temperature": ...}`, and look up the right dictionary by category. I can set that up if you'd like.

Temperature conversions
PY



Do the same with this one:

I'll build this the same way as the length converter, with 11 units and all 121 pairs written out, and compare the results against the numbers in your screenshot.

I've written `area_conversions.py`, which has the full 11×11 dictionary: all 121 conversions hardcoded, including each unit to every other unit. Converting 12 square meters to each unit matches your screenshot (for example, 129.166925 ft², 14.35188 yd² and 0.0029652646 acres).

The units are keyed in lowercase, such as `"square meter"` and `"hectare"`, and `convert()` lowercases its inputs so the dropdown's capitalized names still work:

python

```python
convert(1, "Acre", "Square Foot")       # 43560.0
convert(1, "Hectare", "Square Meter")   # 10000.0
```

The values come from exact definitions (1 acre = 43,560 ft², 1 ft = 0.3048 m). Rounding the displayed results will hide any small float error.

Area conversions
PY

DO this with all the possible volume conversions

You didn't attach a dropdown this time, so I'll use a standard set of 17 volume units (metric, cubic imperial, and US liquid measures). That makes 289 pairs. If your dropdown has different units, send a screenshot and I'll match it.

I've written `volume_conversions.py`, which has the full 17×17 dictionary: all 289 conversions hardcoded, including each unit to every other unit. The test checks passed (1 gallon = 3.785411784 L, 1 ft^3 ≈ 7.4805 gallons, 1 m^3 = 1000 L).

The 17 units are:

- **Metric:** cubic kilometer, cubic meter, cubic centimeter, cubic millimeter, liter, milliliter
- **Cubic imperial:** cubic mile, cubic yard, cubic foot, cubic inch
- **US liquid:** gallon, quart, pint, cup, fluid ounce, tablespoon, teaspoon

Two things to be aware of:

- **US vs. imperial:** the gallon, quart, pint, cup, fluid ounce, tablespoon and teaspoon use US measures. UK imperial units are different (an imperial gallon is 4.54609 L), so tell me if you want those added.
- **Key names:** the keys are lowercase, such as `"cubic meter"` and `"fluid ounce"`, and `convert()` lowercases its inputs, so dropdown labels like "Fluid Ounce" will match.

python

```python
convert(1, "Gallon", "Liter")          # 3.785411784
convert(2, "Cup", "Tablespoon")        # 32.0
```

If your volume dropdown has different units, such as a barrel or a bushel, send a screenshot or the list and I'll regenerate the file to match.

Volume conversions

Expanded during Deploy assignment:

Files:

final_project_2.py

final_project_2.py278 lines

PY

Secret_page.py

Secret_page.py1 line

PY

Length_Page.py

Length_Page.py136 lines

PY

Volume_Page.py

Volume_Page.py337 lines

PY

Area_Page.py

Area_Page.py155 lines

PY

Temperature_Page.py

Temperature_Page.py31 lines

PY

Failing to run on github

Checking the unit lookup and the server start in CI

I can't see the workflow file or the error log, but your code points to a few likely causes. Here they are, most likely first.

1. `drafter` isn't installed on the runner.
GitHub starts from a blank machine, so without a `pip install` step you get `ModuleNotFoundError: No module named 'drafter'`.

2. `start_server(...)` never finishes.
If the workflow runs `python final_project_2.py`, the web server starts and waits forever,

so the job hangs until it times out. It may also fail trying to open a browser on a machine with no screen. Guard the last line so the file can be imported safely:

python

```python
if __name__ == "__main__":
    start_server(State(1.0, build_units(), 0.0, True, "Meter",
"Kilometer", "", "Length"))
```

3. `Length_Page.py` runs code on import.
Its `print(...)` lines at the bottom aren't inside an `if __name__ == "__main__":` block, unlike your other page files. They won't crash, but they run every time the main file imports it. Indent them under the guard:

python

```python
if __name__ == "__main__":
    print(convert(1, "Mile", "Inch"))
    print(conversions["mile"]["kilometer"])
    print(conversions["foot"]["inch"])
    print(convert(5, "Yard", "Meter"))
```

4. Filename capitalization.
GitHub runs Linux, where `import Length_Page` fails if the file in the repo is `length_page.py`. Windows and Mac don't care, so it works locally but breaks there. Check that all the page files are in the repo with exactly the same capitalization as the imports.

5. Python version.
`list[Symbol]` in your dataclass needs Python 3.9 or newer. An older version on the runner raises a `TypeError`. Pin a newer one in the workflow.

A minimal workflow that avoids all of these:

yaml

None

```
name: Smoke test
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install drafter
      - run: python -c "import final_project_2"
```

If it's still failing, paste the red error lines from the failed step (or your `.yml` file) and I can pin down the exact cause. Those lines will say which of the above it is.

Run git log --oneline --decorate --graph --all -n 50 > _site/dashboard/commits.txt
/home/runner/work/_temp/2ec744eb-624d-42cb-ad26-039f9aff73e3.sh: line 1: _site/dashboard/commits.txt: No such file or directory
Error: Process completed with exit code 1.

That error is separate from the Python issues I listed earlier. The shell can't write `commits.txt` because the `_site/dashboard/` folder doesn't exist yet on the runner. The `>` redirect creates the file but never the directory.

Create the folder first:

yaml

None

```
- name: Generate commit log
  run: |
    mkdir -p _site/dashboard
```

```
    git log --oneline --decorate --graph --all -n 50 >
_site/dashboard/commits.txt
```

If the folder is supposed to be created by an earlier step (a build or copy step), check that the step ran and wrote to the same path. `_site` is relative to the repo root, so a step that changes directory could put it somewhere else.

Because the command uses `--all`, you'll probably also want the full history. By default `actions/checkout` fetches only the latest commit, so the log would show one line and no other branches:

yaml

```
None

- uses: actions/checkout@v4
  with:
    fetch-depth: 0
```

If the next step fails, paste the log lines for it and I'll look.

Run set -euo pipefail
Using CPython 3.12.16
Creating virtual environment at: .venv
Activate with: .venv/bin/activate
Resolved 53 packages in 444ms
Downloading uvloop (4.3MiB)
Downloading kiwisolver (1.4MiB)
Downloading pygments (1.2MiB)
Downloading pydantic-core (2.2MiB)
Downloading mypy (14.8MiB)
Downloading fonttools (5.2MiB)
Downloading ast-serialize (1.3MiB)

Downloading isort (1.6MiB)
Downloading numpy (16.0MiB)
Downloading pillow (6.8MiB)
Downloading matplotlib (9.4MiB)
Downloading drafter (6.1MiB)
 Downloaded ast-serialize
 Downloaded kiwisolver
 Downloaded pygments
 Downloaded isort
 Downloaded pydantic-core
 Downloaded uvloop
 Downloaded fonttools
 Downloaded pillow
 Downloaded drafter
 Downloaded matplotlib
 Downloaded numpy
 Downloaded mypy
Prepared 53 packages in 419ms
Installed 53 packages in 45ms
 + analyze-drafter-site==0.4.1
 + annotated-doc==0.0.6
 + annotated-types==0.8.0
 + anyio==4.16.0
 + ast-serialize==0.13.1
 + astroid==4.3.4
 + bakery==1.0.0
 + click==8.5.1
 + contourpy==1.4.0
 + coverage==7.17.3
 + cycler==0.12.1
 + dill==0.4.1
 + drafter==2.2.1
 + fonttools==4.68.0
 + h11==0.16.0

+ httptools==0.9.0
+ idna==3.20
+ isort==9.0.2
+ jinja2==3.1.6
+ kiwisolver==1.5.1
+ librt==0.16.0
+ markdown-it-py==4.3.0
+ markupsafe==3.0.4
+ matplotlib==3.11.2
+ mccabe==0.7.0
+ mdurl==0.1.2
+ mypy==2.4.1
+ mypy-extensions==1.2.0
+ numpy==2.5.4
+ packaging==26.3
+ pathspec==1.1.1
+ pillow==13.0.0
+ platformdirs==4.14.5
+ pydantic==2.14.3
+ pydantic-core==2.50.4
+ pygments==2.21.0
+ pylint==4.1.3
+ pyparsing==3.3.3
+ python-dateutil==4.0.0
+ python-dotenv==1.2.4
+ pyyaml==6.0.3
+ rich==15.0.0
+ shellingham==1.5.4
+ starlette==1.10.0
+ tomlkit==0.15.2
+ typer==0.29.1
+ typing-extensions==4.16.0
+ typing-inspection==0.4.5
+ tzdata==2026.6

+ uvicorn==0.54.0
+ uvloop==0.23.0
+ watchfiles==1.3.0
+ websockets==17.4
error: No solution found when resolving dependencies
cause: Because length-page was not found in the package registry and you require length-page, we can conclude that your requirements are unsatisfiable.
Error: Process completed with exit code 1.

Weighing ways to stop local modules being installed

The earlier steps worked. Drafter installed fine, and the commit log error is behind you. This failure comes from the dependency step. It scans your imports and tries to install each one from PyPI, and it treats `import Length_Page` as a package called `length-page`. That package doesn't exist, because it's one of your own files. `Temperature_Page`, `Area_Page` and `Volume_Page` would hit the same problem once that one is resolved.

You have two options.

Option 1: merge everything into `final_project_2.py` (my recommendation).
Drafter sites usually get built and hosted by a script that treats your project as a single file. Merging avoids this error and also any problem with the extra files not being copied into the build. It's simple:

1. Paste the four conversion tables and their `convert()` functions into the main file.
2. Rename them so they don't collide, since all four define `convert` and three define `conversions`:
   - `length_conversions`, `temperature_conversions`, `area_conversions`, `volume_conversions`
   - `convert_length`, `convert_temperature`, `convert_area`, `convert_volume`
3. Delete the four `import ..._Page` lines.
4. Update the two lookup dictionaries near the top:

python

Python

```
UNIT_TABLES = {
    "Length": length_conversions,
```

```
    "Temperature": temperature_conversions,
    "Area": area_conversions,
    "Volume": volume_conversions,
}

CONVERTERS = {
    "Length": convert_length,
    "Temperature": convert_temperature,
    "Area": convert_area,
    "Volume": convert_volume,
}
```

The rest of your code doesn't need to change. Drop the `print(...)` test lines from `Length_Page.py` instead of copying them.

Option 2: keep the files separate.
This only works if the workflow lets you list local modules to ignore, or lets you supply your own requirements. Open the workflow `.yml` (or the script it calls) and look at the step that runs `set -euo pipefail`. If it's a course or template workflow, check its README for how multi-file projects are handled. Don't try to add `length-page` as a dependency, since that name isn't on PyPI.

I can do the merge for you if you'd like. It's a long file because of the conversion tables. Tell me whether you want it as one `.py` file.

+ 1 chat with a TA

+ 